

What a Web Page Is Made Of and How Browsers Display It

A web page begins its life as a text file written in HTML, consisting of structured tags that specify the content rather than the way it looks. When you open an .html file in a browser, the browser goes through the file line by line, interprets each tag and creates in its mind a structure known as the DOM (Document Object Model). It then applies default styling since it knows that headers should be larger, links should be blue, and paragraphs should have margins, after which it produces the actual pixels that you see on your screen. Metadata tags such as `<meta charset="UTF-8">` inform the browser about how to decode the text, while `<meta viewport>` instructs it to adjust the display for use on mobile devices. The browser also downloads any linked materials (such as images or stylesheets if they are included) and shows them in sequence. That is the reason why semantic structure is important: the tags `<header>`, `<nav>`, `<main>`, and `<footer>` are more than just labels; they indicate to assistive technology and search engines what each section is meant to represent.

Three Places I Chose Semantic Tags Over Divs

Originally in `index.html` I replaced the `div` with an `<article>` to enclose the chef's quote. An `<article>` represents self-contained, reusable content — the quote from the chef in effect forms its own unit of meaning. Using a `div` would have meant that this feature would not have been picked up by screen readers and search engines. The `blockquote` within it also serves to indicate the quoted text in a semantic way.

Secondly, in `menu.html` I created an actual `<table>` using `<caption>`, `<thead>`, `<tbody>`, `<th scope="col">`, and `<th scope="rowgroup">`. Although I could have simulated a table using nested `divs` and CSS, that would have been accessibility sabotage. The table I constructed makes it clear that 'the first row contains the headers', 'the columns are for price and item' and 'the rows represent groupings' — machines are able to read it, export it and sort it, and screen readers can navigate it. In contrast, the `divs` would have remained silent.

Third, in `order.html`, I used `<fieldset>` and `<legend>` to group the different sections of the form—namely "Personal Details" and "Order Parameters". I linked each `<label for="username">` to the corresponding `<input id="username">`. When a sighted person taps an input field, a mobile user or a user with a screen reader taps the label and as a result the input becomes active. Using `divs` would have broken that connection.

What Happens When a Visitor Presses the Submit Button

If a person clicks "Тапсырысты Растай" (Confirm Order) in `order.html`, the browser carries out HTML5 validation first: it verifies that the name, email, and phone have been filled in, that the email has the correct format (as specified by `type="email"`), and that a date has been entered in the date field. In the case of failed validation, the browser displays error messages and the form does not then get submitted. If the validation is successful, the form tries to submit. The form tag has `action="#"` and `method="post"` — the # indicates that it currently goes nowhere (there is no server yet, as I explained in my code comments). The

method="post" means that the data would be sent as part of a POST request and would not appear in the URL. In a real deployment, the user's name, phone number, chosen dish, delivery type, any extras and the comments would be sent to a server script which would then process the order. For now, the form resets once it appears to have submitted and no data is stored. As my code comment states clearly, "Form processing functionality is omitted at this stage as server-side script integration is scheduled for future assignment modules."